

BitPoker

Trust-Minimised Heads-Up Poker over Mental Cryptography and On-Chain Adjudication

Technical Whitepaper — Revision 1.0

August 2026

Contents

Abstract	3
1. The problem	3
2. System architecture	5
3. Cryptographic foundations	7
4. The game protocol	8
5. Transport: relays, not direct peer-to-peer	9
6. The chain: matchmaking, escrow, lifecycle	10
7. Deterministic on-chain adjudication	12
8. Relay economics	14
9. Strategies against real-world difficulty	16
10. Verification	20
11. Clients	20
12. Limitations and open problems	21
13. Roadmap	22
14. Conclusion	22
Appendix A — Glossary	23
Appendix B — Source references	24

Abstract

BitPoker is a two-player poker system in which no server ever holds the deck, no operator ever holds the money, and no human referee ever decides a disputed hand. The deck is shuffled jointly by the two players using the Barnett–Smart mental-poker construction over the FourQ elliptic curve, with a Neff-style verifiable-shuffle NIZK proving that each shuffle is a genuine permutation-and-re-encryption of the deck it received. Money is escrowed by a Cosmos SDK application chain (`pokerchain`) at matchmaking time and released either by a *cooperative settlement* — both players signing the same outcome — or, when cooperation fails, by a **deterministic C++ adjudication engine executed inside consensus by every validator**, which replays the hand from the players’ own signed message history and their revealed per-hand secrets and computes a verdict that is a pure function of on-chain state.

Gameplay traffic itself flows through economically-incentivised relays, because two consumer-grade endpoints behind NAT cannot reliably reach each other. Relays are paid out of a rake taken from the pot itself — never from block subsidy — and are compensated only on presentation of *bilateral adversarial receipts*: a signature from each of the two mutually hostile players. Relay network locations are not published on chain at all; a relay learns it has been assigned a session by watching consensus, and answers with an ECIES-sealed blob containing a per-player one-time connect token.

This document describes the cryptographic construction, the protocol and its wire formats, the chain module, the adjudication engine and its three-layer determinism guard, the relay economics, and — at some length — the concrete failures encountered when this design met real networks, real gas limits and real CLI tools, and the strategies adopted in response. Where a problem remains open, it is stated as open.

1. The problem

Online poker asks players to trust a house with three separate powers: the power to shuffle, the power to see, and the power to pay. Historically, all three have been abused. The 2007–2008 insider “superuser” scandals were failures of the second power. Sudden operator insolvency is a failure of the third. And the first power — the shuffle — is unauditably by construction in a client-server design: the player receives cards and a promise.

Decentralising this is not simply a matter of putting poker on a blockchain. Three constraints collide:

1. **Cards are secret, but fairness must be public.** A blockchain is a broadcast medium with no private state. Naively putting the deck on chain reveals it to everyone, including the opponent.
2. **Poker is latency-sensitive and high-frequency.** A heads-up hand involves dozens of protocol messages. Putting every action on chain costs a block interval per action and is economically absurd at micro-stakes.
3. **The interesting cases are the failures.** Any two honest players can settle by agreement. The entire design burden falls on what happens when one player disconnects mid-hand, refuses to reveal, or submits a forged proof — precisely when the money is still in play.

BitPoker’s answer is a *layered* one. Cryptography removes the need to trust the shuffle. A fast off-chain channel carries the hand. The chain holds the money and serves as the court of last resort — but only enters when cooperation fails, and when it does, it decides by *recomputation*, not by arbitration.

1.1 Design goals

- **No trusted dealer.** Neither player, nor the relay, nor any validator can learn a card that the protocol does not entitle them to, nor bias the shuffle.
- **No trusted arbiter.** Dispute resolution is a deterministic function of chain state plus submitted evidence — reproducible bit-for-bit on every validator, on every CPU architecture.
- **Escrow can always be released.** No sequence of crashes, disconnects or refusals may leave funds permanently locked. Every session state has an exit.
- **Fault is attributable.** Where an outcome depends on someone's misbehaviour, the protocol identifies *whose*, and prices it.
- **The honest path is cheap.** Two cooperating players pay two transactions — one to open an intent, one to settle — plus a small relay rake.

1.2 Non-goals

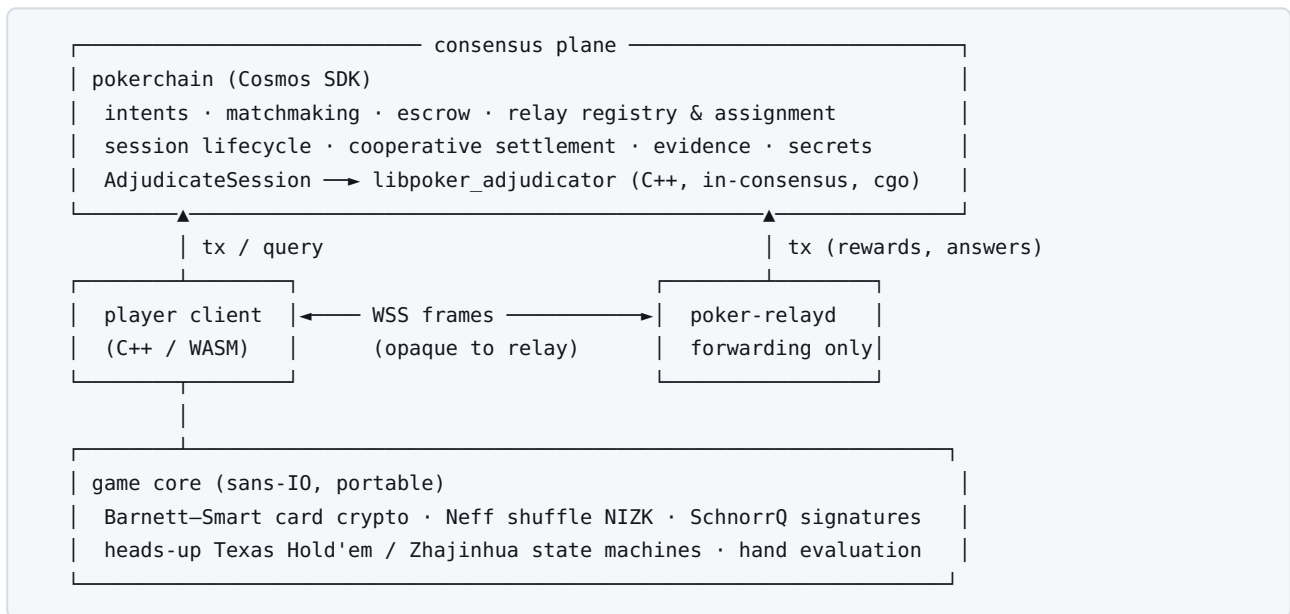
- **Multiway tables.** Everything here is heads-up (exactly two seats). This is not incidental: the adversarial-receipt construction (§8.1) and the two-party shuffle both depend on the “exactly two mutually hostile parties” structure.
- **Preventing real-time collusion.** Two humans sharing a voice channel cannot be stopped by cryptography; this is out of scope, as it is for every poker system.
- **Hiding player identity.** Chain accounts are pseudonymous but linkable.

1.3 Threat model

Adversary	Capability assumed	Countered by
Malicious opponent	Full control of their own client; may forge messages, abandon mid-hand, refuse to reveal	Shuffle NIZK, per-message signatures, on-chain adjudication with forfeit-on-non-reveal
Malicious relay	Sees all ciphertext frames; may drop, delay, reorder, or refuse service	Cards never decryptable by relay; assignment is consensus-determined; non-service is visible as an unanswered assignment; reconnection + resync
Malicious validator	Participates in consensus; may run modified software	Adjudication is a pure function; divergence is a consensus fault, and the cross-architecture determinism gate makes accidental divergence detectable in CI
Network adversary	Observes and interrupts transport	WSS with on-chain public-key pinning; protocol-layer relay-hello signatures; bounded-silence detection escalating to the chain
Griefer	Wastes an honest player's time and money without directly profiting	Dispute fee charged to the at-fault side; escrow timeouts with refund paths; relay assignment-miss accounting

2. System architecture

BitPoker is four cooperating planes, each with a different trust and performance profile.



2.1 The three consumer boundaries

The C++ codebase (~51k lines, with ~39k lines of Go on the chain side) is partitioned not by topic but by *who links it*. This turned out to be the single most consequential structural decision in the project, because it is what makes the consensus-critical path defensible.

- **bitpoker_gamecore** — cryptographic primitives, card/stack types, and the sans-I/O subset of each game (`*_hand_core`, `*_rank`, `*_state`, `*_protocol`). Zero `boost::asio`, zero wallet code, zero I/O.
- **bitpoker_core** — links gamecore, and adds the asio coroutine session layer (`common/`), Cosmos signing (`wallet/`), chain and relay clients, and the `*_game.*` files that are the only ones permitted to touch asio.
- **poker_adjudicator** — the C ABI shared by Go validators. It links **bitpoker_gamecore only** — deliberately *not* **bitpoker_core**.

The rule for placing a new file is mechanical: if `grep -l boost/asio` matches, it belongs in the transport layer; otherwise in the core. A CMake boundary check (`test/check_gamecore_boundary.cmake`) enforces it. The payoff is that a future contributor cannot pull a non-deterministic dependency into the consensus path by adding a single `#include`, because the link would fail.

The same partition is what lets three very different consumers exist:

Consumer	Boundary	Links
pokerchaind validators	C ABI via cgo (<code>binding/ adjudicator</code>)	gamecore only
Keplr browser wallet	Emscripten embind (<code>wasm/</code>)	gamecore only; JS drives the session
Native clients (Qt desktop, Android, TUI, CLI, dispatcher)	direct C++ (<code>client/</code>)	full core

3. Cryptographic foundations

3.1 Card representation and joint keys

Each of the 52 cards is a point on the FourQ curve; a *stack* is an ordered vector of ElGamal-style ciphertext pairs under a **combined public key** formed by adding the two players' session public key points. Neither player's key alone decrypts anything; both key shares are needed, and each share is released only for the specific card the protocol says may be seen.

A player's session identity is a `FiatShamirSecretKey` — a FourQ scalar plus its derived 96-byte public commitment (`publicKey || commitment || random`). This is a **per-session game key, entirely separate from the player's chain account key**. The separation matters for the dispute path: revealing the session secret key after a hand exposes that hand's cards — which are being adjudicated anyway — and nothing else. Funds are never at risk from a reveal.

The 96-byte public key is committed on chain in the player's `GameIntent` before any cards exist, which is what makes the later revelation checkable: the engine verifies `session_secret_key · G == the on-chain public key point` before it believes anything a disputing party submits.

3.2 The shuffle

Shuffling happens exactly twice per hand, once by each player, in sequence:

1. Player A takes the canonical initial stack, applies a secret permutation π and re-encrypts every card with fresh randomness r_x under the combined key, producing stack S_1 plus a NIZK proof P_1 .
2. Player B verifies P_1 against $(S_0 \rightarrow S_1)$, then applies its own π' and r_x' , producing S_2 and P_2 . S_2 is the deck the hand is played from.

The proof (`NeffVerifiableMixing::ProveStack / VerifyStack`) establishes that the output stack is a permutation-and-re-encryption of the input stack **without revealing which permutation**. Because the combined key is the *sum* of the two public key points, the construction is symmetric: it does not matter who shuffles first, and the verification does not depend on shuffle order — a property the adjudication engine relies on when it reconstructs the shuffle chain from evidence that may be partial.

The security argument for the deck is then simply: for the deck to be biased, a player would have to produce a shuffle that is *not* a permutation-and-re-encryption and still pass the NIZK; and for a card to be read early, an adversary would need both key shares.

3.3 Revealing a card

To let the opponent see card i , a player emits a `FiatShamirCardkey` — a per-card decryption share with an attached proof that the share was correctly derived from the player's committed public key and that specific ciphertext. The receiver validates the share against the sender's public key before applying it; an invalid share is a hard cryptographic fault, not a retry.

Combining one's own share with the peer's yields the plaintext card point, which maps back to a face value via the canonical stack-zero table.

There is one subtlety with large consequences. The normal decryption helper path draws randomness internally. The adjudication engine must never draw randomness (§7.2), so it decrypts by a mathematically equivalent but RNG-free route: direct elliptic-curve point subtraction using the two known session secret keys. Same result, zero entropy consumed.

3.4 Message authentication

Every protocol message (`MsgMove`) carries a `MsgHeader` with a session id, a message id, the previous message id, type flags, and a **SchnorrQ signature over the whole serialised message with the signature field zeroed**. Signing uses the session key, not the chain key. Two properties follow:

- The message history is a **signed, hash-linked chain**. Neither party can later present a rewritten history: the adjudicator attributes every action to whichever seat’s session key signed it, and uses that attribution — not the submitter’s claims — to enforce turn order.
- Evidence is self-authenticating. A player submits *the opponent’s own signed messages* as evidence; the opponent cannot repudiate them.

Message ids are structured as `(hand_id << 16) | round_id`, which gives cheap ordering, duplicate detection and hand-boundary checks (`protocol_state.hpp`) without extra state.

4. The game protocol

4.1 Hand lifecycle

The base framework (`P2PPokerGameBase` / `P2PPokerHandBase`) drives every game type through a C++20 coroutine pipeline over Boost.Asio, so asynchronous network code reads like straight-line logic:

```
runGame
└─ per hand: runHand
    │   ├── exchangeShuffleStack    (two shuffles + two NIZK verifications)
    │   ├── runGameLogic           ← game-specific; the only virtual step
    │   └─ settleHand              (signed settlement transcript)
```

Concrete games implement `runGameLogic()` and `getWinAmount()`. Three exist:

- **CC (Simple Card Compare)** — a minimal reference game used to exercise shuffle, reveal and settlement. It is *rejected by the chain* for real-money intents, because the adjudication engine has no replay path for it; it survives as a development and test vehicle.
- **ZJH (Zhajinhua / “Three-Card Brag”)** — three cards per player, look-or-blind betting, comparison-based showdown.
- **TH (Heads-up Texas Hold’em)** — the primary game: preflop / flop / turn / river / showdown, with check, call, bet, raise, fold and all-in, blinds derived from the effective stake, and a 7-choose-5 evaluator covering all ten hand ranks including the wheel straight and royal flush.

4.2 Sans-IO state machines

For TH and ZJH the *rules* live in pure functions with no I/O and no allocation of network concepts:

```

HeadsUpHoldemState      create_heads_up_holdem_hand(...);
HeadsUpHoldemActionResult apply_heads_up_holdem_action(HeadsUpHoldemState&, action);
bool                    heads_up_holdem_hand_complete(const HeadsUpHoldemState&);
void                    settle_heads_up_holdem_showdown(HeadsUpHoldemState&, int cmp);

```

This is not merely tidy. **The adjudication engine replays disputes through the exact same functions the live game uses.** There is deliberately no second implementation of “the rules, as understood by the referee” — the classic source of arbitration systems that disagree with the game they arbitrate. When the engine rules an action illegal, it is illegal by the same code that would have rejected it at the table.

4.3 Settlement transcript

At the end of a hand the two clients exchange a `MsgSettle` carrying both seats’ amounts, both continue-flags, and *both* signatures. The transcript that gets signed (`SettlementTranscriptContext`) binds: a domain string and schema version, the chain session id, the final stake, both addresses, the winner/split outcome, the hash of the final settle message — and, critically, the **relay fee raked on chain** together with the net per-seat amounts after that rake.

That last part is a lesson learned rather than an obvious design: if players sign the pre-rake game distribution, they are signing something the chain will not actually pay, and any later reconciliation is ambiguous. The transcript signs the true on-chain payout.

5. Transport: relays, not direct peer-to-peer

5.1 Why relays

The original design was direct peer-to-peer. In practice, two consumer endpoints — mobile networks with carrier-grade NAT, corporate firewalls, browser clients with no listening socket at all — cannot reliably establish a direct connection, and a browser-based wallet client *never* can. A relay is a forwarding node: it sees frames, forwards them, and cannot read cards, because everything meaningful is encrypted under a key the relay does not possess.

Relays are therefore an availability component, not a trust component. The security consequences of a fully malicious relay are limited to denial of service — and denial of service by a relay is exactly the situation the escrow-timeout and dispute paths were built to survive.

5.2 Connection establishment

1. Both players open on-chain intents; the chain matches them, escrows both stakes, creates a `GameSession`, and **deterministically assigns** a primary relay plus up to two backups by weighted hash (§8.4).
2. The assigned relay observes the new session and answers with `MsgProvideRelayEndpoint` (§8.3), carrying one ECIES-sealed blob per player.

3. Each client decrypts *its own* blob to obtain a `wss://` URL, a one-time connect token, and an expiry height, then opens the WebSocket presenting the token in `Sec-WebSocket-Protocol` — the only header a browser can set.
4. The client authenticates with a signed `ClientHello` (`cosmos-signature-v1`), which the relay checks against on-chain session authorisation.
5. The peers exchange `MsgSessionHello` (display name, session pubkey, chain session id) and begin the hand.

TLS uses on-chain public-key pinning for native clients: the certificate's raw public key must equal the `Relay.Pubkey` registered on chain, so a self-signed certificate is sufficient and the trust anchor is consensus, not a CA. Browsers cannot pin, so relays serving browsers must present a real CA certificate; the protocol-layer signed `relay-hello` covers relay impersonation in both cases.

5.3 Reconnection and resynchronisation

Frames carry a per-sender sequence number reused from the relay frame header's `request_id`. On reconnect, a client sends `SessionResume` with the highest sequence it received; the peer retransmits everything newer and replies with its own high-water mark so the exchange is symmetric. Receivers deduplicate by sequence, so overlapping retransmission is harmless. A `MSGTYPE_RESYNC` mid-hand resynchronisation lets an interrupted hand continue rather than escalate.

The relay itself contributes two behaviours discovered to be necessary: a reconnecting client with the same player id **displaces** the half-open old connection (otherwise the ghost holds the slot), and an overflowing outbound queue **disconnects the slow consumer** rather than silently dropping frames (a silently dropped frame desynchronises the protocol in a way that looks like cheating).

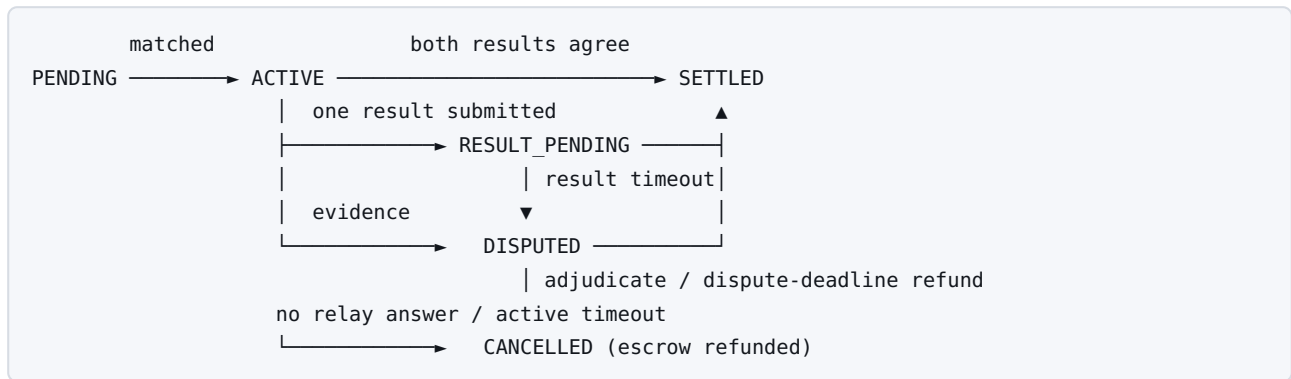
6. The chain: matchmaking, escrow, lifecycle

`pokerchain` is a Cosmos SDK application chain. Its base denomination is `uchip` (exponent 0) with `CHIP` as the display denomination at 10^6 .

6.1 Intent and matching

`MsgOpenGameIntent` carries the game type, stake, optional named opponent, the player's 96-byte session public key, and a 33-byte compressed `secp256k1` *transport* public key (used for the ECIES endpoint blob). Opening an intent locks the stake into escrow immediately. Compatible pending intents are matched in-consensus, which creates a `GameSession`, assigns relays, snapshots the relay fee, and pseudo-randomly assigns seats by session hash — so neither player chooses the button.

6.2 Session lifecycle



Every state has a bounded exit, which is a property the design had to acquire the hard way (§9.1). Governance parameters control the bounds:

Parameter	Default	Meaning
<code>result_timeout_blocks</code>	20	How long a single submitted result waits for the peer's matching result
<code>dispute_timeout_blocks</code>	50	After which <i>anyone</i> may crank adjudication of a disputed session
<code>session_active_timeout_blocks</code>	10000	Backstop for sessions that never play
<code>intent_timeout_blocks</code>	720	How long a pending intent stays matchable
<code>dispute_fee_bps</code>	100 (1%)	Fee charged to the at-fault side (ADR-004)
<code>relay_fee_flat / relay_fee_bps / relay_fee_cap</code>	0	Ceilings on relay self-quoted pricing (ADR-007 P3)
<code>relay_min_bond</code>	100 000 000	Minimum relay stake
<code>relay_claim_window_blocks / relay_answer_timeout_blocks</code>	0 (off)	Endpoint-answer window arithmetic (ADR-007 P1)

6.3 Cooperative settlement

The happy path is `SubmitSessionResult` from both seats. When the two submissions agree, `settlePot` releases escrow — paying each seat its final chip stack, with the relay rake deducted — and the session becomes `SETTLED`. Conservation ($\text{payA} + \text{payB} + \text{fees} == \text{pot}$) is asserted inside the keeper; a bank transfer failure rolls back the entire settlement rather than recording a verdict whose money never moved.

6.4 Escalation

If cooperation fails, either player submits `SubmitSessionEvidence` (the signed message history plus context) — the first such submission moves the session to `DISPUTED` — followed by `SubmitSessionSecret` (the 32-byte session scalar, checked against the committed 96-byte public key). Then anyone may call `AdjudicateSession`.

7. Deterministic on-chain adjudication

This is the heart of the system, and its hardest engineering constraint.

7.1 Why determinism is the whole problem

Deciding *who won a hand of poker* is easy — the rules are mechanical. The difficulty is that in a dispute, the chain holds only what the parties submitted, and **every validator must independently derive byte-identical results from that input**. Any divergence is not a bug; it is a consensus fork that halts the chain. The “state transition function” here is an entire poker rules engine plus elliptic-curve verification, which is orders of magnitude more complex than a transfer or a delegation.

The chosen approach (ADR-003, “Scheme A”) is to implement the engine once in C++, expose it through a pure C ABI, and link it into the validator binary via `cgo`, so every validator executes the identical implementation.

Two integration decisions guard the boundary:

- The Go side is decoupled at a `[]byte → []byte` interface. Without the `cgo_adjudicator` build tag the adjudicator is `nil` and `AdjudicateSession` returns `ErrAdjudicationUnavailable` — a misconfigured pure-Go node **fails loudly instead of silently returning a default verdict**.
- Every C ABI entry point is wrapped in a catch-all. A C++ exception unwinding into Go is undefined behaviour, and a crash inside consensus halts the chain; exceptions are mapped to deterministic status codes at the boundary.

7.2 The three-layer determinism guard

Layer 1 — runtime sentinel. The invariant is: *the adjudication path must never consume cryptographic randomness*. Shuffle *proving* needs an RNG; shuffle *verifying* does not, but both live in the same translation unit, so a compile-time split is not available. Instead, a `thread_local` armed/disarmed flag with an RAII scope guard (`DeterministicGuardScope`, `nestable`) is armed on entry to adjudication, and every actual entropy source — `generate_random_with_openssl<T>()`, `RandomKey()`, `generateRandomIV()` — calls `checkDeterministicGuard()` and **throws** if armed. Non-reproducible output is converted from a silent consensus hazard into a loud, deterministic failure.

Layer 2 — structural isolation. As described in §2.1, the adjudicator target links `bitpoker_gamecore` only. Non-deterministic dependencies are not merely discouraged; they are unlinkable.

Layer 3 — cross-architecture differential CI. “Byte-identical” is not a claim that survives on code review. A frozen corpus of eight real `DisputeAdjudicationRequest` payloads covering every verdict path is replayed in CI on both `ubuntu-24.04 (x86_64)` and `ubuntu-24.04-arm (arm64)`; each run emits `<name> <abi-status> <sha256(result-bytes)>`, and the workflow explicitly diffs the two architectures against a committed golden digest. Any divergence fails the gate. In practice this has held across compiler and optimisation changes — `gcc-15 -O0` debug rebuilds reproduce goldens generated by `gcc-16 -O2`.

Finally, an `EngineVersion` string (currently `0.2.0`) is stamped into every verdict. Any change to adjudication behaviour must bump it, and is treated as consensus-relevant — requiring a coordinated chain upgrade.

7.3 The adjudication pipeline

```

parse request
  → preflight: context / key / signature validation
  → shuffle chain reconstruction + NIZK verification
  → betting replay through the live state machine
  → decrypt & rank
  → allocate pot / assign fault
  → serialise result

```

Preflight validates both session public keys, checks `secret_key · G == authoritative public key point`, and attributes every message in the history to a seat by signature. An inconsistent secret does not fail the request; it marks `secret_valid = false`, which the verdict stage treats as a forfeit.

Shuffle verification extracts the two shuffles, forms the combined key by point addition (order-independent), and verifies `before → after` stepwise, returning `Verified / Incomplete / BadShuffle`.

Betting replay runs the recorded actions through the same state machine the live game uses, with seat attribution taken from preflight signatures so that turn-order violations are caught by the engine's own `OutOfTurn` check.

Decrypt and rank is possible because the adjudicator holds *both* session secret keys, so it can decrypt the verified stack directly without any submitted per-card keys.

Allocation ignores the claimed dispute reason entirely. The engine does not trust the requester's stated grievance; it derives the outcome from evidence, in short-circuit priority order:

1. Hard cryptographic violation (bad shuffle)
2. Hard logic violation (illegal action)
3. Fold already resolved the hand
4. Requires decrypt-and-compare — including forfeit for missing/invalid secrets, and refund when the hand never dealt out

Pot allocation first returns uncalled excess bets, then splits the matched portion by the comparison result; an odd chip on a split goes to the big-blind seat, matching the live settlement path exactly.

7.4 Verdicts, faults and fees

Verdicts are `FIRST_PLAYER_WINS`, `SECOND_PLAYER_WINS`, `SPLIT`, `REFUND`; result codes distinguish `SHOWDOWN`, `FORFEIT_NO_SUBMISSION`, `FORFEIT_INVALID_SECRET`, `VIOLATION_BAD_SHUFFLE`, `VIOLATION_ILLEGAL_ACTION`, `BOTH_FORFEIT`, `REFUND_UNDEALT`. Fault is `NONE`, `FIRST`, `SECOND` or `BOTH`.

The dispute fee (ADR-004) is charged **only when fault is single-sided** — from the winner's payout, routed to the standard Cosmos fee collector, so it reaches validators through the same pipe as gas. No fee is charged when fault is `NONE` (a normal showdown or fold is not misconduct) or `BOTH` (each side simply gets its stake back). Pot conservation is unaffected: the fee changes the split, not the total.

The honest assessment recorded in ADR-004 is that **the 1% default has no quantitative justification**. Too high, and an honest player who merely had the bad luck of a cheating opponent is taxed for it; too low, and it fails to deter. It is a governance parameter awaiting real data.

7.5 Who triggers adjudication

Either participant may trigger it at any time. After `dispute_timeout_blocks`, **anyone** may — a permissionless crank. This closes the fund-lock risk without requiring the at-fault party’s cooperation.

Automatic adjudication in `EndBlock` was deliberately *not* built. If a validator without the cgo engine participated in an `EndBlock` that calls it, the result would diverge and halt the chain. The permissionless crank achieves the same liveness property — the winner and any watcher are both motivated to call it — at far lower risk.

8. Relay economics

A relay costs money to run and, in the base design, earned nothing. Solving that requires answering a question the decentralised-CDN/VPN literature treats as open: **how do you prove a packet was forwarded?**

8.1 Service proof: bilateral adversarial receipts

Purely cryptographic delivery proof is impossible — a relay can claim to have forwarded, and a player can claim not to have received. BitPoker sidesteps this using a structural lever: **a session has exactly two players, they are mutual adversaries, both have chain identities, and both have money escrowed.**

Each client signs a receipt over the canonical bytes

```
bitpoker-relay-receipt-v2\n<chain_id>\n<session_id>\n<relay_id>
```

and hands it to the relay over the session channel. The relay collects **both**, then submits `MsgClaimRelayReward`. The chain verifies both signatures, that the signers are exactly this session’s two players, that the relay is one the chain itself assigned, and that the session is `SETTLED`.

This is not proof of delivery. It is a structure in which **forging a claim requires bribing your opponent** — a single honest player can veto any false claim about their own session.

8.2 Self-funded rake: why there is no subsidy

The reward is raked from the pot at matchmaking time and paid at settlement. It is **never** minted or subsidised. That single choice removes the dominant attack: a Sybil operator running a relay and two sock-puppet “players” against each other merely moves money from their own player accounts to their own relay account, while paying gas — a guaranteed net loss. No staking or reputation system is required for this argument to hold; it is structural.

Because the fee is already sunk into escrow at matchmaking, signing a receipt costs the player *nothing marginal*, so there is no rational reason to withhold it. (A spiteful player can still withhold to grief the relay; the relay loses one flat fee and can deprioritise that player. A transcript-countersigning fallback is designed but not implemented.)

Attack	Defence
Relay claims for a session it never served	Needs both adversaries' signatures
Relay colludes with one player	The other player simply does not sign
Sybil self-play to farm fees	Self-funded rake makes it a net loss after gas
Claiming for an unassigned session	Chain checks <code>relay_id</code> against its own assignment
Serving half a session then vanishing	Reward requires the session to reach <code>SETTLED</code>
Fabricating a huge pot	Pricing is flat + capped bps

8.3 Endpoint privacy: the assignment–answer protocol

Publishing relay endpoints on chain publishes a target list. ADR-007 removes `endpoint` from the on-chain `Relay` record entirely — the public key stays (it is the trust anchor), only the network location is hidden.

The replacement is an **assignment–answer protocol**:

1. The chain assigns relays as before (deterministic weighted hash — this *must* stay in the keeper; letting proposers choose relays would let them route sessions to their own).
2. Assigned relays watch for new sessions. `MsgProvideRelayEndpoint` carries two blobs, each ECIES-sealed to that player's per-intent transport public key (`secp256k1 ECDH` → `HKDF-SHA256` → `AES-256-GCM`, `AAD` = `"adr007-v1" || session_id || relay_id` to prevent cross-session replay). Each blob holds a URL, a **per-player one-time connect token**, and an expiry.
3. Failover is arbitrated by **on-chain windows**, not by guesswork: the primary has an exclusive window $[H, H+W)$, backup i unlocks at $H+(i+1)W$, and the **first accepted answer wins**. Without this rule, two relays could each allocate resources and each attract one player.
4. If every window expires unanswered, the session is voided and both escrows refunded.
5. **Liveness is redefined as real work**. With endpoints hidden there is nothing to probe, so synthetic challenge/response is abandoned: *failing to answer an assignment is itself on-chain evidence of failure*. Misses are counted, decay over time, reduce assignment weight, and accumulate toward slashing.

The honest limit, stated in the ADR rather than glossed: an adversary can still farm endpoints by opening intents. Mitigations make it expensive (rotating frontend pools, token gating that drops unauthenticated traffic at the first packet, and the stake + gas cost of every intent) but do not make discovery impossible. The goal is **pricing discovery, not preventing it**. Volumetric L3/L4 attacks remain a network-layer problem solved with bandwidth, anycast or CDN fronting — the protocol cannot help there, and the document says so.

Live testnet verification of this protocol found a real bug that all three language-specific unit-test suites had missed: a legacy check treated an empty registry endpoint as an error and killed sessions *after* successful blob decryption. That is the characteristic value of end-to-end testing — each side was individually green.

8.4 Bond and pricing

Bond buys two concrete things, which is what makes staking more than a ritual:

- **Assignment weight** $1000 \cdot \sqrt{\text{bond} / \text{min_bond}}$ — sublinear, so a large operator gets more work but cannot buy a proportional monopoly.

- **Concurrency capacity** — in-flight sessions are capped at `bond / relay_per_session_collateral`, enforced *at answer acceptance* (the moment work is actually taken on), so bond is genuine per-session collateral.

Bond may be topped up instantly and partially unbonded; a queued tranche stops supporting weight and capacity immediately but **remains slashable**, and slashing draws from the active bond first and the queue second. Without that ordering, a misbehaving relay could simply front-run punishment by unbonding.

Pricing is a **capped market**: relays quote their own `fee_flat / fee_bps`, governance sets ceilings, and the quote feeds assignment weight (a zero quote keeps full weight; quoting at the ceiling halves it). Halving rather than zeroing is deliberate — price should *tilt* the lottery, not decide it, or a single operator undercutting to zero recreates the winner-take-all outcome the market was meant to avoid. The session snapshots **the selected relay's quote** at creation, so neither a price change nor a governance vote can reprice a session whose escrow is already locked.

Adjudicated sessions pay the relay too — deducted first from forfeited escrow when there is a single at-fault party. The reasoning is behavioural: if a session that ends in arbitration pays the relay nothing, relays learn to avoid exactly the players who most need a relay. Voided hands (refunds, expired-dispute refunds) pay nothing, since both sides are restored to their prior position and there is no forfeited escrow to draw from.

8.5 Relay identity

A relay's identity is rooted in a persistent ed25519 node key (`relay_node_key.json`), from which the relay id is derived: `bech32("relay", sha256(pubkey)[:20])`. The id becomes a *public-key commitment* rather than a name: anyone can verify `relay_id == derive(pubkey)`, ids cannot collide, restarts are idempotent, and the same key signs the `/status/signed` endpoint and (in `auto TLS` mode) the WSS leaf certificate. The economic identity (the Cosmos account that posts bond and receives rewards) stays deliberately separate from this operational identity.

9. Strategies against real-world difficulty

Everything above is design. What follows is what happened when it met reality — included because the failures were more instructive than the successes, and because most of them were invisible to unit tests.

9.1 Sessions that could never end

Symptom. A live test run left three matched sessions parked in `ACTIVE` forever, with escrow locked. `ClaimSessionTimeout` only accepted `RESULT_PENDING`; becoming `ACTIVE` set no deadline at all. If both clients crashed, never started, or simply walked away, nothing on chain could ever move that session.

Strategy. Treat “every state has a bounded exit” as an invariant to be audited per state, not an emergent property. `session_active_timeout_blocks` and the unanswered-assignment void path close the two known dead ends. The general lesson is that in an escrow system, the state diagram must be read as a *liveness* diagram: a state with no outgoing timed edge is a fund-loss bug even if no code path is wrong.

9.2 The disconnect that never disconnects

Symptom. The single most damaging bug found. A player’s peer process was killed mid-shuffle; the survivor hung for **over ten minutes** with no reconnect, no evidence submission, nothing. The reason is structural: the client’s WebSocket connects to *the relay*, not to the peer. When the peer’s process dies, that connection stays perfectly healthy — the relay answers pings normally. Beast’s `timeout::suggested` covers connection idleness, not “connection fine, peer has stopped talking.” The design document’s assumption that “resync relies on the transport read timeout to establish that the peer never returned” was simply false under relay transport: **that timeout did not exist.**

The dispute pathway was, in other words, unreachable. Everything downstream — adjudication, dispute fees, forfeit verdicts — had been unit-tested against a gate no live game could ever open.

Strategy. `readPeerMessageBounded` caps the wait on the peer specifically, using a budget derived as $2 \times \text{action_timeout} + 30$ (default 150 s). Timeout is handled on the same path as a read failure: reconnect → resync → escalate on chain. The resync handshake read is bounded too, or the handshake hangs the same way.

The budget’s *derivation* matters more than its value. It must be substantially larger than the opponent’s own action timeout — otherwise a slow but honest opponent gets classified as absent, and a real-money game is escalated into a dispute. **Falsely escalating an honest game is worse than waiting longer**, so the parameter is derived from the action timeout rather than set independently, making it impossible to configure the two into contradiction.

Result: ten-plus minutes of hanging became 60 s detection → reconnect → bounded resync → on-chain escalation.

9.3 Empty evidence, and the temptation to fix it on chain

Symptom. When the peer vanishes *very* early — before the survivor has recorded any message at all — the survivor’s history is empty. The chain rejects empty evidence (`evidence_payload` has no message history), the follow-up secret is rejected too (“secrets are only accepted for a disputed session”), and the session sits in `ACTIVE` until the ~14 h backstop.

The tempting fix and why it was rejected. Relaxing the on-chain gate to accept “connection lost” evidence with no history would let *either* party pull a live session into `DISPUTED` with a content-free submission. Empty history is not “undecidable” to the engine — it is a *valid verdict*, `REFUND_UNDEALT`, which refunds each side’s on-chain starting chips. Since sessions are multi-hand and the chain records the *opening* stacks, a player several hands down could submit empty evidence, immediately adjudicate, and **erase their losses**. Plugging that would require further consensus rules (“empty history may not be adjudicated before the dispute deadline”) — two interlocking rules in consensus for a situation the client can avoid entirely.

Strategy. Fix it in the client, leave the consensus gate intact. If `collectMessageHistory()` is empty, the client submits neither evidence nor secret — the latter especially, since that transaction is certain to be rejected while still publicly burning the session identity — and marks the session `abandonedBeforePlay()`. The session runner then drives the pre-existing *retreat* path (split-pot abort result → result timeout → `DISPUTED` → reveal → adjudicate), reaching a refund in about **six minutes instead of fourteen hours**, using only existing primitives and **zero consensus changes**.

Placing this in the shared session runner rather than in each UI was equally deliberate: unattended clients (the dispatcher worker) have nobody to press a “retreat” button, and they are precisely the ones most likely to face an opponent who never arrives.

This fix exposed a further latent defect: the retreat path had only ever supported *native* signing. Clients that sign via the external `pokerchaind` CLI — which hold a key name and a keyring, not an unlocked keystore — failed **every** recovery transaction with `native signing requires an unlocked keystore`. The visible “retreat” button in the UI had never worked for that entire class of client. Only running the path for real revealed it.

9.4 Waiting for an opponent who may legitimately be slow

Symptom. If the peer dies before sending its session hello, the survivor blocks forever in `initSession` — worse than §9.2, since it never even reaches “session failed.”

Strategy, with an explicit trade-off. The §9.2 budget was *not* reused, because waiting for a hello is also a **legitimate recovery state**: the protocol supports both parties re-announcing to resume a session, and a player may return much later. A flat 150 s would break “keep the table open while my opponent reconnects.” So the bound is long and configurable — `peerHelloTimeoutSeconds`, default **1800 s**, `0` meaning wait forever — with a per-second countdown surfaced through the event sink to every front end (Qt, mobile, TUI render it; the CLI prints at 30 s intervals and every second for the last ten, because per-second terminal output is noise). On expiry the client does *not* retry the relay — that only re-spends the same budget — but reuses the §9.3 conclusion: no messages means no dispute, so go straight to the refund path. Verified live at 2.5 minutes end to end with the bound lowered to 60 s.

9.5 Transactions that “succeeded”

Three independent failure modes, all of which made the client believe it had reached the chain when it had not:

- **Gas that does not scale with payload.** Evidence carries an entire hand’s message history; a single shuffle message is ~16 KB. One real hand needed **732 070** gas against a fixed 300 000 shared by all game transactions — meaning *every real evidence submission was guaranteed to fail*, and to fail **after** `CheckTx` passed, so the client saw success. Fixed by scaling: `400 000 + 60 × payload_bytes`.
- **`pokerchaind tx` exits 0 on rejection.** The CLI returns success even when the node rejects the transaction, so code 32 (sequence mismatch) and code 11 (out of gas) were both read as success. The submitter now parses the response `code`, and retries sequence mismatches with a bounded wait of 6 s — longer than one 5 s block, because 2 s never once worked.
- **Broadcast accepted ≠ executed.** Acceptance into the mempool says nothing about inclusion or execution. The submitter now polls `query tx <hash>` for inclusion *and* `code == 0`. This immediately surfaced two previously invisible classes of failure: transactions silently dropped from the mempool (how the secret submissions were being lost) and transactions included but failing execution (the out-of-gas evidence).

Strategy. Treat every layer boundary as an adversary that lies about success — exit codes, mempool acceptance, and `CheckTx` are all *not* proof of execution. Confirm at the layer that actually matters, and size resources from payload rather than from a constant.

9.6 Silent capability gaps between client classes

Relay rewards never arrived under CLI signing mode. Signing a receipt requires the account private key *inside the client process*; clients that delegate signing to an external CLI keep the key in a keyring and therefore **silently never sent receipts**. The relay simply waited forever, and no one was told. Both sides now warn (the client explains the cause and remedy; the relay reports `n/2 receipts` at session close). The proper fix — arbitrary-payload signing in `pokerchain-signer` — remains open.

The general strategy: when a capability is optional per client class, its absence must be *reported*, not skipped. A silently-skipped feature is indistinguishable from a broken one and costs far more to diagnose.

9.7 Tooling that lies

- **cgo stale archives.** Go’s build cache does not hash C++ archive *contents*, so validators kept linking a stale adjudicator. A deliberate `ArchiveHash ldflags` cache-buster forces relinking.
- **A CLI descriptor that panicked every subcommand.** `autocli` permits exactly one optional positional argument, and it must be last; adding three made `pokerchaind` panic on *every* subcommand including `init` — while `go test ./...` stayed entirely green. Fixed by moving to flags, plus a root-command guard test verified to catch the regression.
- **A rake charged on refunds.** `REFUND` verdicts were still taking a relay fee, while three other void paths were not — the same user-visible outcome was charged or not purely by which code path reached it. Fees are now taken only when the hand truly produced a result.

Each of these was found by end-to-end runs, not unit tests. The standing conclusion is recorded in the project’s own notes: three green unit suites prove three components work in isolation and say very little about the seam between them.

9.8 Making disputes reproducible at all

The dispute end-to-end scenario could not be tested, because a scripted hand runs from shuffle to settlement in about **200 ms** — there is no wall-clock window in which to kill a peer. Attempts at 12 s, 4 s, 3 s and 1.2 s all landed after cooperative settlement.

Strategy: deterministic fault injection at protocol granularity, not wall-clock.

`BITPOKER_FAULT_DROP_AFTER_FRAMES=n` makes a client `_Exit` immediately *before* emitting frame *n*, so failures can be placed at exact protocol positions (frame 1 = announce, 2 = session hello, 3 = the first gameplay frame; hence `n=3` is “handshake complete, nothing shuffled”). Combined with seat lookup — seats are assigned pseudo-randomly on chain, so injecting into “the second client” hits the wrong seat roughly half the time — this made the entire dispute pathway reproducible on demand.

10. Verification

Layer	Mechanism
C++ unit tests	~491 tests across core crypto, protocol, games, chain, relay, adjudicator, client, robot
Go unit tests	46 test files covering keeper, params, genesis, relay daemon, protocol
Determinism gate	Frozen 8-case corpus replayed on x86_64 + arm64, digests diffed against goldens
Cross-language golden vectors	Relay receipt canonical bytes (C++ / Go), relay-id derivation (daemon / keeper), ECIES endpoint blob (Go / C++ / JS), ClientHello signing (JS signed, Go verified)
End-to-end matrix	18 TH/ZJH scenarios against a live chain + relay + two clients, with per-hand settlement audits reconciling gross, rake and balances
Live testnet	Full dispute chain verified on chain: silence detection → reconnect → resync → evidence → DISPUTED → adjudicate → engine verdict → escrow released

The cross-language golden vectors deserve emphasis. Wherever the same bytes must be produced by C++, Go and JavaScript — receipts, derivations, sealed blobs — the contract is pinned by a vector one implementation generated and the others verify. Three implementations that each pass their own tests will otherwise drift, and the drift only appears in production.

11. Clients

One core, five front ends, all driving the same session logic:

- **CLI** (`app/main.cpp`) — a thin REPL over `bitpoker_client` ; the development reference.
- **Qt desktop** — five screens with an animated felt table for TH and ZJH.
- **Android** — Qt/QML mobile build, shipped through M0–M6.
- **TUI** (`bitpoker-tui`) — terminal wallet + table, with an intent watcher that auto-joins matched sessions; the usual host for unattended play.
- **Keplr browser extension** — the WASM path: `gamecore.wasm` via embind, with JavaScript driving the session, an intent-approval popup, lobby and table UX. This client never links the asio transport or the wallet layer — Keplr provides its own signing stack.

Two further components share the same core: a **dispatcher** that routes intents to worker clients, and a **robot player** — a personality-driven bot for TH and ZJH, built on a sans-IO `DecisionView` seam so the brain is testable without a network.

12. Limitations and open problems

Stated plainly, because a whitepaper that only lists strengths is marketing.

Unilateral disputes without peer awareness. A player may, mid-game, submit their genuine history as evidence, pull the session into `DISPUTED`, and reveal their own secret. An opponent not watching the chain will be judged to have forfeited for failing to reveal, and can lose the entire escrow. The intended fix — clients polling their own session status during play and reacting immediately to `DISPUTED` — is designed but not yet implemented. This is the most significant known gap.

Multi-hand settlement granularity. The chain records opening stacks and settles a session as a whole; per-hand checkpointing is designed (`M3`) but unbuilt. Until then, “sit down and play N hands” is constrained, and adjudication reasons about a hand within a session rather than a running stack.

Bad-signature violations are not a verdict. A history containing a non-participant’s signature fails preflight as `MALFORMED_REQUEST` rather than producing a `VIOLATION_BAD_SIGNATURE` verdict, because the on-chain model has no “accuser” field. Tampering currently causes failure, not attributable loss.

Collusive “friendly” disputes. Two colluding players can both decline to submit, producing a `BOTH` refund and avoiding all fees. Whether this warrants economic suppression is undecided.

The 1% dispute fee is unmodelled (§7.4).

Timeout policy is implicit. When a hand times out mid-street but both players reveal, the engine decrypts the full board and runs a complete 7-choose-5 showdown — “running it out” for cards never dealt in play. This is a fairness-relevant policy choice currently documented only in implementation comments; it deserves its own ADR.

Browser TLS. Self-signed certificates with on-chain pinning work only for native clients. Any relay serving browsers needs a real CA certificate; ACME automation is designed but not implemented.

Volumetric DDoS. Endpoint privacy raises the cost of discovery; it does not address L3/L4 capacity attacks, whose only real answer is network-layer budget.

Relay griefing. A player who withholds their receipt costs the relay one flat fee with no on-chain consequence. The transcript-countersignature fallback that would close this is specified but unbuilt.

13. Roadmap

Milestone	Content	Status
M1	Dispute loop end-to-end on a live chain: session key plumbing, validator cgo builds, adjudication transaction, dispute scenarios	Complete
M2	Full normal-play regression on chain, multi-relay failover, resync verification	Largely complete
M3	Exact final-stack settlement, multi-hand checkpointing, bad-signature verdicts, differential fuzzing of the determinism guard	Open
M4	Production deployment: genesis and upgrade handlers, container images with the cgo engine, monitoring runbooks	Partially complete
M5	Relay rewards R0–R1 (receipts, rake, bond, lease, challenges, weighting)	Complete
ADR-007 P1–P4	Endpoint privacy, self-selected bond, fee market, operations documentation	Complete and verified on a local testnet

Immediate priorities are the unilateral-dispute watcher (§12), arbitrary-payload signing so CLI-mode clients can issue relay receipts (§9.6), and per-hand checkpointing.

14. Conclusion

BitPoker’s central claim is narrow and, we think, defensible: **a two-player card game can be made trust-minimised end to end without putting the game on chain.** Cryptography handles the deck. A relay handles reachability without handling secrets. The chain handles money and, only on failure, judgment — and it judges by recomputing the hand from the players’ own signed words, identically on every validator.

The parts of this that were hard were not the parts that look hard. The Barnett–Smart construction and the shuffle proof are textbook. What took real work was: guaranteeing bit-identical execution of a poker engine inside consensus across architectures; discovering that a relay-mediated connection gives no signal at all when the peer dies; realising the dispute pathway had been carefully built behind a door that could never open; and resisting the temptation to fix client problems by adding rules to consensus.

The recurring principle across all of them is that **failure modes deserve at least as much design as the happy path** — and that they must be reproducible on demand, or they will only be discovered by users, with money on the table.

Appendix A — Glossary

Term	Meaning
Barnett–Smart	Mental-poker construction giving jointly-encrypted, jointly-shuffled cards decryptable only with both parties' key shares
FourQ	High-performance elliptic curve used for all in-game cryptography
SchnorrQ	Signature scheme over FourQ, used to sign every protocol message
NIZK shuffle proof	Non-interactive zero-knowledge proof that a stack is a permutation and re-encryption of another, revealing nothing about the permutation
FiatShamirKey	96-byte in-game session public key (<code>publicKey commitment random</code>)
FiatShamirCardkey	Per-card decryption share with a validity proof
Session identity	Per-session game keypair, distinct from the chain account key
Intent	On-chain offer to play, which escrows the stake
Evidence	Signed message history submitted to move a session to <code>DISPUTED</code>
Secret	The revealed session secret scalar enabling adjudicated decryption
Verdict / Fault	Engine outputs: the payout decision, and who (if anyone) caused it
Bilateral adversarial receipt	Signatures from both opposing players proving a relay served their session
Assignment–answer	Protocol by which an assigned relay privately delivers its endpoint plus per-player connect tokens
Determinism guard	Runtime sentinel forbidding RNG use inside the adjudication path

Appendix B — Source references

Area	Location
Cryptographic core	<code>bitpoker/include/core/</code> , <code>bitpoker/src/core/</code>
Session and transport	<code>bitpoker/{include,src}/common/</code> , <code>.../relay/</code>
Games	<code>bitpoker/{include,src}/texasholdem/</code> , <code>.../zhajinhua/</code>
Adjudication engine	<code>bitpoker/{include,src}/adjudicator/</code> , <code>binding/adjudicator/</code>
Chain module	<code>pokerchain/x/pokerchain/</code>
Relay daemon	<code>pokerchain/cmd/poker-relayd/</code> , <code>pokerchain/x/pokerchain/relay/</code>
Wire formats	<code>proto/bitpoker/{game,relay,adjudication}/v1/</code> , <code>pokerchain/proto/</code>
Clients	<code>bitpoker/app/{qt,mobile,tui,dispatcher}/</code> , <code>bitpoker/wasm/</code> , <code>keplr-wallet/</code>
Design records	<code>bitpoker/docs/adr/ADR-003 ... ADR-007</code> , <code>bitpoker/docs/ONCHAIN_ROADMAP.md</code>
Relay operations	<code>docs/relay/</code> — deployment (nginx + TLS) and operations
Explorer	<code>docs/explorer/</code> — node prerequisites and nginx deployment